| EX NO:5 | DEVELOP SPRING MVC APPLICATIONS INTEGRATED WITH A BACKEND DATABASE |
|---|---|
| DATE: | |

**AIM:**

To develop Spring MVC applications integrated with a backend database.

**QUESTION 1:**

Create a Spring MVC application that integrates Spring AOP for logging and security, using the following packages:
Entity: Bank Account Management
Packages & Classes
• com.kce.entity
- Define a BankAccount entity (accountNumber, accountHolder, balance, accountType).

• com.kce.util
- Implement HibernateUtil for session management.

• com.kce.dao
- Create BankAccountDAO for CRUD operations.

• com.kce.aop
- Implement an AOP aspect to log method execution times.
- Add a security aspect to restrict access to certain bank accounts based on user roles.

• com.kce.controller
- Implement BankAccountController to handle requests and forward to JSPs.

Views (JSP pages)
• Add/Edit Bank Account – Form to add/edit account details.
• List Bank Accounts – Displays accounts with role-based access control.
• Bank Account Details – Shows account details with logging applied.

**CODE:**

**com.kce.bankaccountmanagement**
**BankAccountManagementApplication**

```
package com.kce.bankaccountmanagement;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BankAccountManagementApplication {
	public static void main(String[] args) {
		SpringApplication.run(BankAccountManagementApplication.class, args);
	}
}
```

**com.kce.bankaccountmanagement.aop**
**LoggingAspect.java**

```
package com.kce.bankaccountmanagement.aop;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;
@Aspect
@Component
public class LoggingAspect {
   @Around("execution(* com.kce.bankaccountmanagement.dao.*.*(..))")
   public Object logTime(ProceedingJoinPoint joinPoint) throws Throwable {
      long start = System.currentTimeMillis();
      Object result = joinPoint.proceed();
```

```
        long end = System.currentTimeMillis();
        System.out.println(joinPoint.getSignature()+" executed in "+(end-start)+"ms");
        return result;
    }
}
```

**SecurityAspect.java**

```
package com.kce.bankaccountmanagement.aop;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;
@Aspect
@Component
public class SecurityAspect {
    @Before("execution(* com.kce.bankaccountmanagement.dao.*.*(..))")
    public void checkSecurity() {
        System.out.println("Security check passed");
    }
}
```

**com.kce.bankaccountmanagement.controller**

**BankAccountController.java**

```
package com.kce.bankaccountmanagement.controller;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import com.kce.bankaccountmanagement.dao.BankAccountDAO;
import com.kce.bankaccountmanagement.entity.BankAccount;

@Controller

public class BankAccountController {

    @Autowired
    BankAccountDAO dao;

    @GetMapping("/")
    public String home(Model model) {

        List<BankAccount> accounts = dao.findAll();

        model.addAttribute("accounts", accounts);

        return "listAccounts";
    }

    @GetMapping("/add")
    public String addForm(Model model) {

        model.addAttribute("account", new BankAccount());

        return "addAccount";
    }
```

```
    @PostMapping("/save")
    public String saveAccount(@ModelAttribute BankAccount account) {

        dao.save(account);

        return "redirect:/";
    }

    @GetMapping("/details/{id}")
    public String details(@PathVariable Long id, Model model) {

        BankAccount account = dao.findById(id).orElse(null);

        model.addAttribute("account", account);

        return "accountDetails";
    }
}
```

**com.kce.bankaccountmanagement.dao**
**BankAccountDAO.java**

```
package com.kce.bankaccountmanagement.dao;
import org.springframework.data.jpa.repository.JpaRepository;
import com.kce.bankaccountmanagement.entity.BankAccount;
public interface BankAccountDAO extends JpaRepository<BankAccount, Long> {
}
```

**com.kce.bankaccountmanagement.entity**
**BankAccount.java**

```
package com.kce.bankaccountmanagement.entity;
import jakarta.persistence.*;
@Entity
@Table(name="bank_account")
public class BankAccount {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long accountNumber;
    private String accountHolder;
    private double balance;
    private String accountType;
    public Long getAccountNumber() {
        return accountNumber;
    }
    public void setAccountNumber(Long accountNumber) {
        this.accountNumber = accountNumber;
    }
    public String getAccountHolder() {
        return accountHolder;
    }
    public void setAccountHolder(String accountHolder) {
        this.accountHolder = accountHolder;
    }
    public double getBalance() {
        return balance;
    }
```

```
    public void setBalance(double balance) {
        this.balance = balance;
    }
    public String getAccountType() {
        return accountType;
    }
    public void setAccountType(String accountType) {
        this.accountType = accountType;
    }
}
```

**com.kce.bankaccountmanagement.util**
**HibernateUtil.java**

```
package com.kce.bankaccountmanagement.util;
public class HibernateUtil {
}
```

**accountDetails.jsp**

```
Account Number: ${account.accountNumber}
<br>

Holder: ${account.accountHolder}
<br>

Balance: ${account.balance}
<br>

Type: ${account.accountType}
```

**addAccount.jsp**

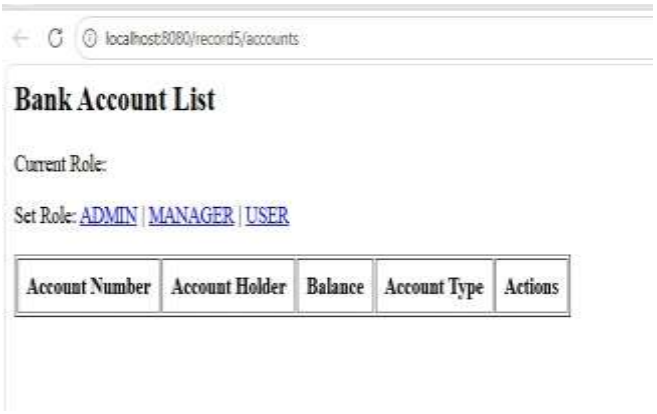
```
<form action="save" method="post">
Account Holder:
<input type="text" name="accountHolder"/>
Balance:
<input type="text" name="balance"/>
Account Type:
<input type="text" name="accountType"/>
<input type="submit" value="Save">
</form>
```

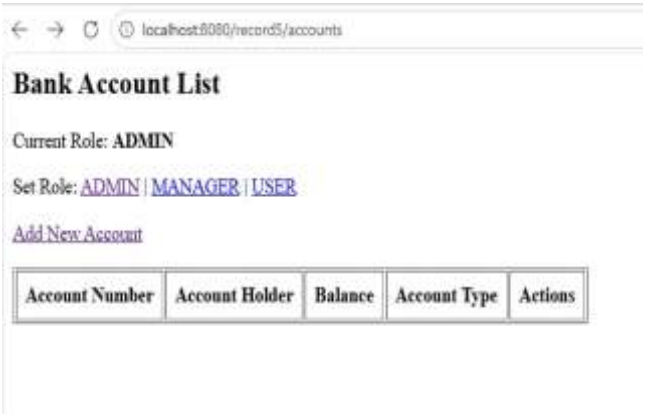
**listAccounts.jsp**

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<a href="add">Add Account</a>
<table border="1">
<tr>
<th>ID</th>
<th>Holder</th>
<th>Balance</th>
<th>Type</th>
<th>Details</th>
</tr>
<c:forEach var="a" items="${accounts}">
<tr>
<td>${a.accountNumber}</td>
<td>${a.accountHolder}</td>
```

```
<td>${a.balance}</td>
<td>${a.accountType}</td>
<td>
<a href="details/${a.accountNumber}">View</a>
</td>
</tr>
</c:forEach>
</table>
```

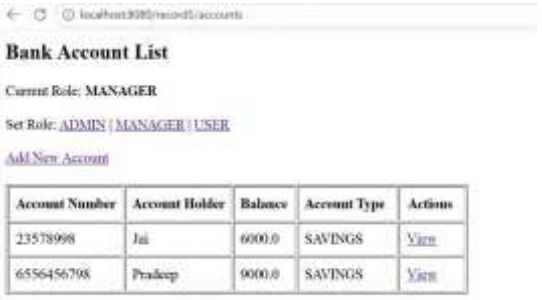
**OUTPUT:**









**QUESTION 2:**

Create a Spring MVC application that integrates Spring DAO and Transaction Management using the following packages:

Entity: Library Management System

Packages & Classes

• com.kce.entity

o Define a Book entity (bookId, title, author, genre, availableCopies).

• com.kce.util

o Implement HibernateUtil for session management.

• com.kce.dao

o Create BookDAO using Spring DAO and Hibernate.

o Implement transaction management for borrowing and returning books.

o Ensure rollback on failed transactions.

• com.kce.controller

o Implement BookController to handle requests and forward to JSPs.

Views (JSP pages)

• Add/Edit Book – Form to add/edit book records.

• List Books – Displays books with transaction handling for availability.

• Book Details – Shows individual book details.

**CODE:**

**com.kce.librarymanagement**

**LibraryManagementApplication.java**

```
package com.kce.librarymanagement;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class LibraryManagementApplication {
	public static void main(String[] args) {
		SpringApplication.run(LibraryManagementApplication.class, args);
	}
}
```

**com.kce.librarymanagement.controller**

**BookController.java**

```
package com.kce.librarymanagement.controller;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.ui.Model;
import  org.springframework.web.bind.annotation.*;
import  com.kce.librarymanagement.dao.BookDAO;
import com.kce.librarymanagement.entity.Book;

@Controller

public class BookController {

@Autowired
BookDAO dao;

@GetMapping("/")
public String listBooks(Model model){
```

```
List<Book> books = dao.findAll();

model.addAttribute("books",books);

return "listBooks";
}

@GetMapping("/add")
public String addBookForm(Model model){

model.addAttribute("book",new Book());

return "addBook";
}

@PostMapping("/save")
public String saveBook(@ModelAttribute Book book){

dao.save(book);

return "redirect:/";
}

@GetMapping("/details/{id}")
public String bookDetails(@PathVariable Long id,Model model){

Book book = dao.findById(id).orElse(null);

model.addAttribute("book",book);

return "bookDetails";
}

@Transactional
@GetMapping("/borrow/{id}")

public String borrowBook(@PathVariable Long id){

Book book = dao.findById(id).orElse(null);

if(book.getAvailableCopies()>0){

book.setAvailableCopies(book.getAvailableCopies()-1);

dao.save(book);
}
return "redirect:/";
}
@Transactional
@GetMapping("/return/{id}")

public String returnBook(@PathVariable Long id){
Book book = dao.findById(id).orElse(null);
book.setAvailableCopies(book.getAvailableCopies()+1);
```

```
dao.save(book);
return "redirect:/";
}
}
```

**com.kce.librarymanagement.dao**
**BookDao.java**

```
package com.kce.librarymanagement.dao;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.kce.librarymanagement.entity.Book;
@Repository
public interface BookDAO extends JpaRepository<Book,Long>{
}
```

**com.kce.librarymanagement.entity**
**Book.java**

```
package com.kce.librarymanagement.entity;
import jakarta.persistence.*;
@Entity
@Table(name="BOOK")

public class Book {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long bookId;

private String title;
private String author;
private String genre;
private int availableCopies;

public Long getBookId() {
return bookId;
}

public void setBookId(Long bookId) {
this.bookId = bookId;
}

public String getTitle() {
return title;
}

public void setTitle(String title) {
this.title = title;
}

public String getAuthor() {
return author;
}

public void setAuthor(String author) {
this.author = author;
```

```
}

public String getGenre() {
return genre;
}

public void setGenre(String genre) {
this.genre = genre;
}

public int getAvailableCopies() {
return availableCopies;
}

public void setAvailableCopies(int availableCopies) {
this.availableCopies = availableCopies;
}
}
```

**com.kce.librarymanagement.util**
**HibernateUtil.java**

```
package com.kce.librarymanagement.util;
public class HibernateUtil {
}
```

**addBook.jsp**

```
<form action="save" method="post">
Title
<input type="text" name="title"><br>
Author
<input type="text" name="author"><br>
Genre
<input type="text" name="genre"><br>
Available Copies
<input type="text" name="availableCopies"><br>
<input type="submit" value="Save">
</form>
```

**bookDetails.jsp**

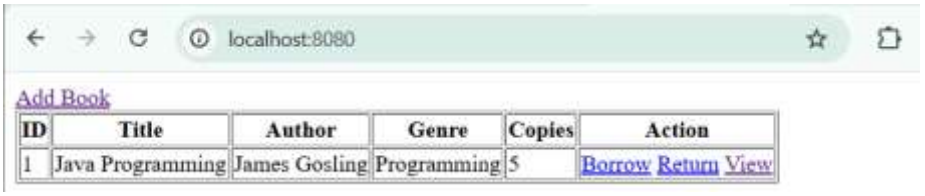
```
Book ID : ${book.bookId} <br>
Title : ${book.title} <br>
Author : ${book.author} <br>
Genre : ${book.genre} <br>
Available Copies : ${book.availableCopies}
```

**listBooks.jsp**

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<a href="add">Add Book</a>
<table border="1">
<tr>
<th>ID</th>
<th>Title</th>
<th>Author</th>
<th>Genre</th>
<th>Copies</th>
```

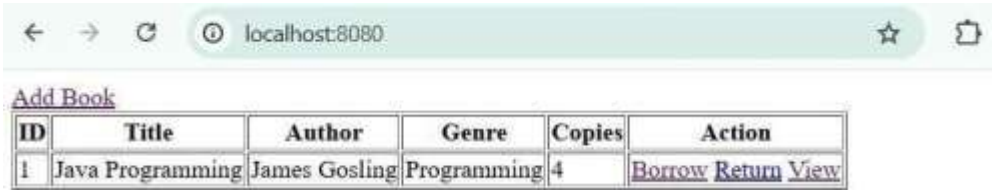
```
<th>Action</th>
</tr>
<c:forEach var="b" items="${books}">
<tr>
<td>${b.bookId}</td>
<td>${b.title}</td>
<td>${b.author}</td>
<td>${b.genre}</td>
<td>${b.availableCopies}</td>
<td>
<a href="borrow/${b.bookId}">Borrow</a>
<a href="return/${b.bookId}">Return</a>
<a href="details/${b.bookId}">View</a>
</td>
</tr>
</c:forEach>
</table>
```

**OUTPUT:**

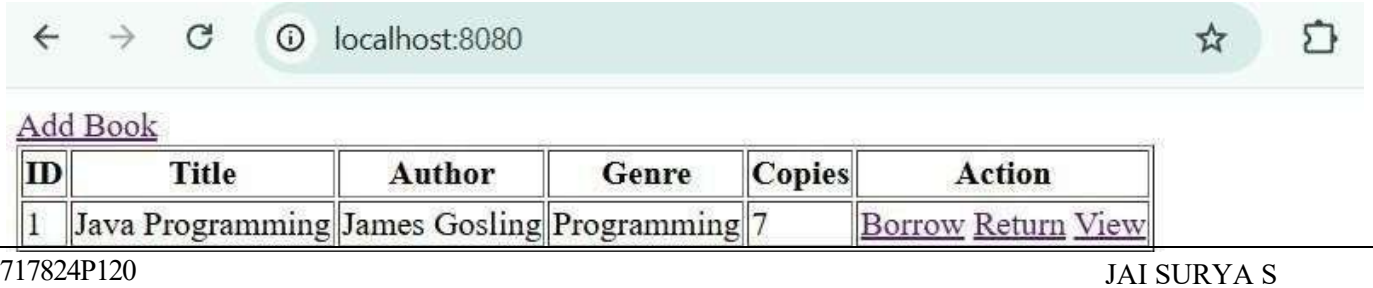



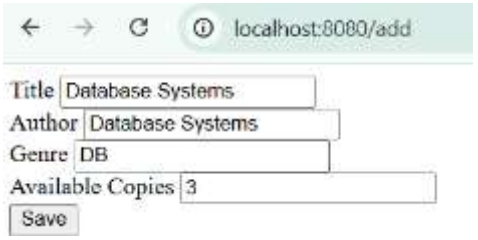

**View**

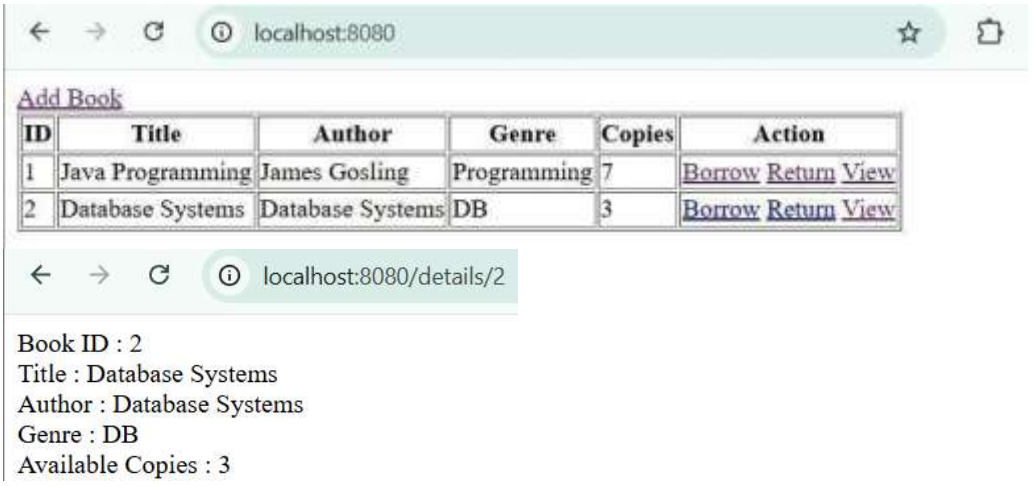

**Borrow**




**Return**









```
BOOK_ID TITLE                AUTHOR                GENRE              AVAILABLE_COPIES
------- -------------------- --------------------- ------------------ ----------------
      1 Java Programming     James Gosling         Programming                       7
      2 Database Systems     Database Systems      DB                                3
```

**RESULT:**

Thus, the development of Spring MVC applications integrated with a backend database has been executed successfully and the output was verified.